

1_sensors

May 22, 2025

1 Comprensión y recopilacion de datos de Sensores

1.1 recopilacion de datos de Sensores

1.1.1 Exploracion inicial de la arquitectura de Datos y Dependencias

Los registros históricos de sensores específicamente para combustible se almacenan en la tabla [jmineops].[rep].[custom_RepLibre_hist_Test_Combustible], generada mediante el procedimiento almacenado [rep].[sp_get_RepLibre_Test_Combustible]. Para identificar las fuentes primarias de datos, se ejecutó el siguiente script SQL que rastrea dependencias del objeto:

-- realizar una busqueda profunda del objeto, buscando donde minimamente fue referenciado el objeto

```
SELECT
    referencing_schema.name AS esquema,
    referencing_obj.name AS objeto,
    referencing_obj.type AS tipo_objeto
FROM
    sys.sql_modules AS mod
JOIN
    sys.objects AS referencing_obj ON mod.object_id = referencing_obj.object_id
JOIN
    sys.objects AS referencing_schema ON referencing_obj.schema_id = referencing_schema.schema_id
WHERE
    mod.definition LIKE '%custom_RepLibre_hist_Test_Combustible%'
```

El sistema Hexagon Mining opera con sensores VIMS (Vital Information Management System) que centralizan datos operativos. Además de un subsistema de geolocalización opera bajo estándar GMS (Geospatial Mining System), registrando coordenadas en latitud/longitud con precisión milimétrica (mm) y altura en centímetros (cm).

Tabla de Sensores:

Sensor ID	Tipo	Medición	Precisión	Observaciones Técnicas
oem.vims.truck.fuel_gauge	Nivel Com-bustible	Porcentaje residual (pct)	$\pm 0.5\%$	Calcula el consumo mediante diferencial de porcentaje en intervalos de 15 segundos.
oem.vims.truck.eng_spd	RPM Motor	Revoluciones por minuto (rpm.5)	Media revolución (± 0.5 rpm)	Error sistemático por resolución del sensor (ej: registro de 1800 rpm = 1800 ± 0.5).

Sensor ID	Tipo	Medición	Precisión	Observaciones Técnicas
oem.vims. truck. ground_spd	Velocidad	Kilómetros por hora (km/h)	± 0.2 km/h	Validado contra sistema GPS integrado en flota.

Los sensores clave identificados son: nivel de combustible, RPM y velocidad. Estos parámetros, junto con los datos GMS, forman la base técnica para el modelo predictivo. La exclusión de otros sensores (como carga útil) se justifica por redundancia operativa, ya que su información está implícita en los ciclos registrados.

1.1.2 Exploración de la base de datos

Si bien los sensores críticos (combustible, RPM y velocidad) se almacenan en `shift_sensors`, su interpretación requiere cruces con tablas adicionales identificadas mediante análisis de dependencias SQL. Estas relaciones permiten contextualizar operativamente las lecturas sensores que se describirán a continuación:

Tabla: `enum_tables`

Columna	Descripción	Ejemplo	Relevancia para el Proyecto
<code>id</code>	Identificador único autoincremental (clave primaria).	1	Clave primaria para identificar cada tipo enumerado
<code>updated_at</code>	Fecha y hora de la última actualización del registro.	2022-11-17...	Ayuda a auditar cambios y mantener trazabilidad
<code>type</code>	Categoría del tipo enumerado (ej: <code>TopographyType</code> , <code>ProjectStatus</code>).	TopographyType	Agrupar los elementos de tipo enumerado para filtros o validaciones
<code>name</code>	Nombre legible del tipo de enumeración.	Road	Facilita visualización y comprensión de los registros
<code>symbol</code>	Nombre clave del tipo de objeto del registro.	road	Usado en lógica de programación o integraciones
<code>attributes</code>	Texto libre con atributos adicionales (no siempre usado).	-	Puede contener metadatos complementarios
<code>ordinal</code>	Valor numérico para orden o jerarquía del elemento.	1	Permite ordenar visualmente o jerárquicamente los registros
<code>deleted_at</code>	Marca de eliminación lógica (si aplica).	NULL	Útil para manejo de soft deletes
<code>visual</code>	Información visual o textual opcional del tipo enumerado.	NULL	Complemento opcional, sin uso directo en análisis principal

Tabla: `equipment`

Columna	Descripción	Ejemplo	Relevancia para el Proyecto
<code>id</code>	Identificador único del equipo.	286	Clave primaria, base para análisis de uso y estado de equipos

Columna	Descripción	Ejemplo	Relevancia para el Proyecto
<code>type</code>	Tipo de equipo.	Truck	Permite clasificar equipos (camión, pala, perforadora, etc.)
<code>name</code>	Nombre o código del equipo.	T-210	Identificador visual del equipo, común en reportes operacionales
<code>created_at</code>	Fecha de creación del registro.	2021-11-10	Útil para trazabilidad y antigüedad del equipo
<code>updated_at</code>	Fecha de última actualización.	2025-02-28	Determina la vigencia del registro
<code>deleted_at</code>	Fecha de eliminación lógica (si aplica).	NULL	Permite exclusión lógica sin pérdida de datos históricos
<code>revision</code>	Número de revisión (puede estar vacío).	NULL	Versionado del equipo o de su configuración
<code>device_id</code>	ID del dispositivo asociado (relación externa).	567	Asociación con sensores u otros dispositivos
<code>status_id</code>	Estado actual del equipo (relación con enum).	111	Permite determinar si está activo, en mantenimiento, etc.
<code>reason_id</code>	Razón del estado actual (ej: mantenimiento, en reserva, etc.).	324	Diagnóstico o causa asociada
<code>lineup_status_id</code>	Estado en la secuencia operativa.	113	Parte del control de asignación de flota
<code>equipment_type_id</code>	Clasificación del equipo, asociado a tabla <code>enum_tables</code> .	2772	Categoriza técnicamente al equipo
<code>activity_id</code>	Actividad actual del equipo (clave externa).	40	Permite identificar su operación actual
<code>activity_start</code>	Inicio de la actividad actual.	2025-02-28	Útil para análisis de duración de actividades
<code>length</code>	Longitud del equipo.	14	Aporta en simulaciones físicas, distribución en el terreno
<code>unit_id</code>	Unidad asignada al equipo (clave externa).	15	Asocia el equipo a una unidad funcional
<code>project_id</code>	Proyecto al que está asignado el equipo.	185	Permite agrupar datos por proyecto
<code>prestart_check</code>	Indicador de verificación previa (1 = sí, 0 = no).	1	Control de seguridad y cumplimiento de protocolos
<code>warnings</code>	Advertencias activas (si las hay).	“ ”	Puede alertar sobre condiciones anómalas

Tabla: `maintenance`

Columna	Descripción	Ejemplo	Relevancia para el Proyecto
id	Identificador único de la orden de trabajo o registro de combustible.	285	Clave primaria
updated_at	Última actualización del registro.	2024-06-18	Control de versiones y actualización de datos
equipment_id	ID del equipo relacionado.	309	Relación directa con tabla equipment
fuel_tank	Capacidad del tanque de combustible (litros).	4650	Base para análisis de autonomía y consumo
fuel_used	Combustible consumido (litros).	0	Métrica clave de eficiencia
engine_hours	Horas acumuladas del motor (texto).	85597	Usado para estimar mantenimiento y vida útil
fuel_gauge	Porcentaje de medidor de combustible.	92	Indicador operacional útil para alertas
engine_hours_updated_at	Fecha de última actualización de las horas del motor.	2025-02-28	Sincronización con operación real
fuel_gauge_updated_at	Fecha de última actualización del medidor de combustible.	2025-02-28	Verifica integridad de la información del sensor
tkph_avg, tkph_max	Datos relacionados a carga por neumáticos (TKPH - ton-km/h).	-	Útil para análisis de neumáticos
tire_time, tire_alarms	Información sobre neumáticos y sus alertas.	-	Apoya análisis de desgaste o problemas
additives, comment	Campos de texto libre, sin descripción específica.	-	Podrían contener detalles adicionales de contexto operativo

Tabla: **sensor_sets**

Columna	Descripción	Ejemplo	Relevancia para el Proyecto
id	Identificador único del conjunto de sensores.	2	Clave primaria
name	Nombre del conjunto de sensores.	Snapshot: Brake Air Pressure	Define la finalidad o categoría del set de sensores
updated_at	Fecha de actualización del set.	2021-12-09 21:32:03.000	Permite validar vigencia de configuración
deleted_at	Marca de eliminación lógica (si aplica).	2021-12-09 21:32:03.000	Trazabilidad de versiones
device_id	ID del dispositivo que agrupa los sensores.	34	Relación con un equipo físico o sensor gateway
save_seconds	Intervalo de tiempo para guardar lecturas.	-	Define frecuencia de muestreo (no disponible en ejemplo)
sensors	IDs de sensores asociados, separados por coma.	117,172,178	Define el conjunto real de sensores incluidos

Columna	Descripción	Ejemplo	Relevancia para el Proyecto
<code>deadband_percent</code>	Porcentaje de cambio mínimo para considerar una lectura como válida.	-	Ayuda a evitar registros redundantes
<code>sensor_set_type_id</code>	Identificador del conjunto de sensores.	2247	Categorización técnica, permite agrupar tipos similares
<code>expire_at</code>	Fecha de expiración del set (si aplica).	NULL	Define validez temporal de la configuración

Tabla: `shift_sensors`

Columna	Descripción	Ejemplo	Relevancia para el Proyecto
<code>id</code>	Identificador único autoincremental del registro (clave primaria).	86233	Clave principal para referenciar registros individuales de sensores
<code>updated_at</code>	Fecha y hora de la última actualización del registro en formato UTC-0.	2022-10-27 14:55:59	Trazabilidad de actualizaciones
<code>created_at</code>	Fecha y hora de creación del registro en formato UTC-0.	2022-10-27 14:55:55	Útil para análisis de secuencia temporal
<code>equipment_id</code>	ID del equipo asociado (relacionado con la tabla <code>equipment</code>).	307	Vincula la medición a un equipo específico
<code>sensor_set_id</code>	ID del conjunto de sensores utilizados.	473	Identifica qué grupo de sensores generó los datos
<code>sensor_alarm_id</code>	ID de la alarma del sensor asociada (si aplica).	NULL	Permite relacionar eventos de alarma
<code>all_values</code>	Indica si se registraron todos los valores esperados del sensor (1 = Sí, 0 = No).	1	Verifica integridad del conjunto de datos
<code>values</code>	Valores crudos de sensores medidos (ej: "507,2265,509").	507,2265,509	Base para cálculos y análisis técnico
<code>configs</code>	Configuración aplicada a los sensores durante la medición (ej: códigos de parámetros).	507,2265,...	Permite reconstruir el contexto de la medición
<code>latitude</code>	Latitud geográfica del equipo en formato entero (dividir entre 1,000,000).	- 75948559	Permite ubicar el equipo espacialmente
<code>longitude</code>	Longitud geográfica del equipo en formato entero (dividir entre 1,000,000).	- 241961690	Permite ubicar el equipo espacialmente
<code>elevation</code>	Elevación del equipo (en metros, ej: 406019 = 406.019 m).	406019	Contexto geográfico para análisis de condiciones del terreno
<code>active_alarms</code>	Lista de alarmas activas separadas por coma.	"21706,18,509"	Detecta condiciones críticas o anómalas durante el turno
<code>speed</code>	Velocidad del equipo en la unidad definida (km/h, mph, etc.).	0	Indica comportamiento operativo

Columna	Descripción	Ejemplo	Relevancia para el Proyecto
<code>snapshot_id</code>	ID del snapshot general de datos.	80331	Permite vinculación con registros históricos
<code>shift_snapshot_id</code>	ID del snapshot específico del turno de trabajo.	NULL	Útil para análisis por turno
<code>alarm_start_id</code>	ID del evento que inició una alarma (si existe).	NULL	Permite trazar origen de condiciones anómalas
<code>engine_hours</code>	Horas acumuladas de motor (ej: 80331 = 8033.1 horas si se divide por 10).	80331	Métrica de uso del equipo
<code>sensor_health</code>	Estado de salud del sensor (ej: porcentaje de integridad o código).	NULL	Indica si el sensor funciona correctamente
<code>heading</code>	Dirección de desplazamiento del equipo en grados (0–360°).	NULL	Útil para análisis de rutas y patrones de movimiento
<code>odometer</code>	Odómetro: distancia total recorrida por el equipo (km, millas, etc.).	NULL	Mide desgaste y uso acumulado

El análisis de las tablas de la base de datos reveló que el modelo predictivo deberá sustentarse en tres variables críticas antes mencionadas de la tabla `shift_sensors`: el nivel de combustible (`sensor_set_id = 481`), registrado cada 30 segundos según la tabla `sensor_sets`; las revoluciones del motor o RPM (`sensor_set_id = 490`), con captura cada 60 segundos; y la velocidad, presente en todos los registros. Estas variables se complementan con metadatos de geolocalización de alta precisión (milimétrica en coordenadas y centimétrica en altitud).

Un hallazgo clave fue la disparidad temporal en la disponibilidad de datos: los registros de combustible están disponibles desde enero de 2024, mientras que los de RPM solo existen a partir de febrero de 2025. Esta conclusión se sustenta en las siguientes consultas técnicas ejecutadas para determinar las fechas mínimas de registro:

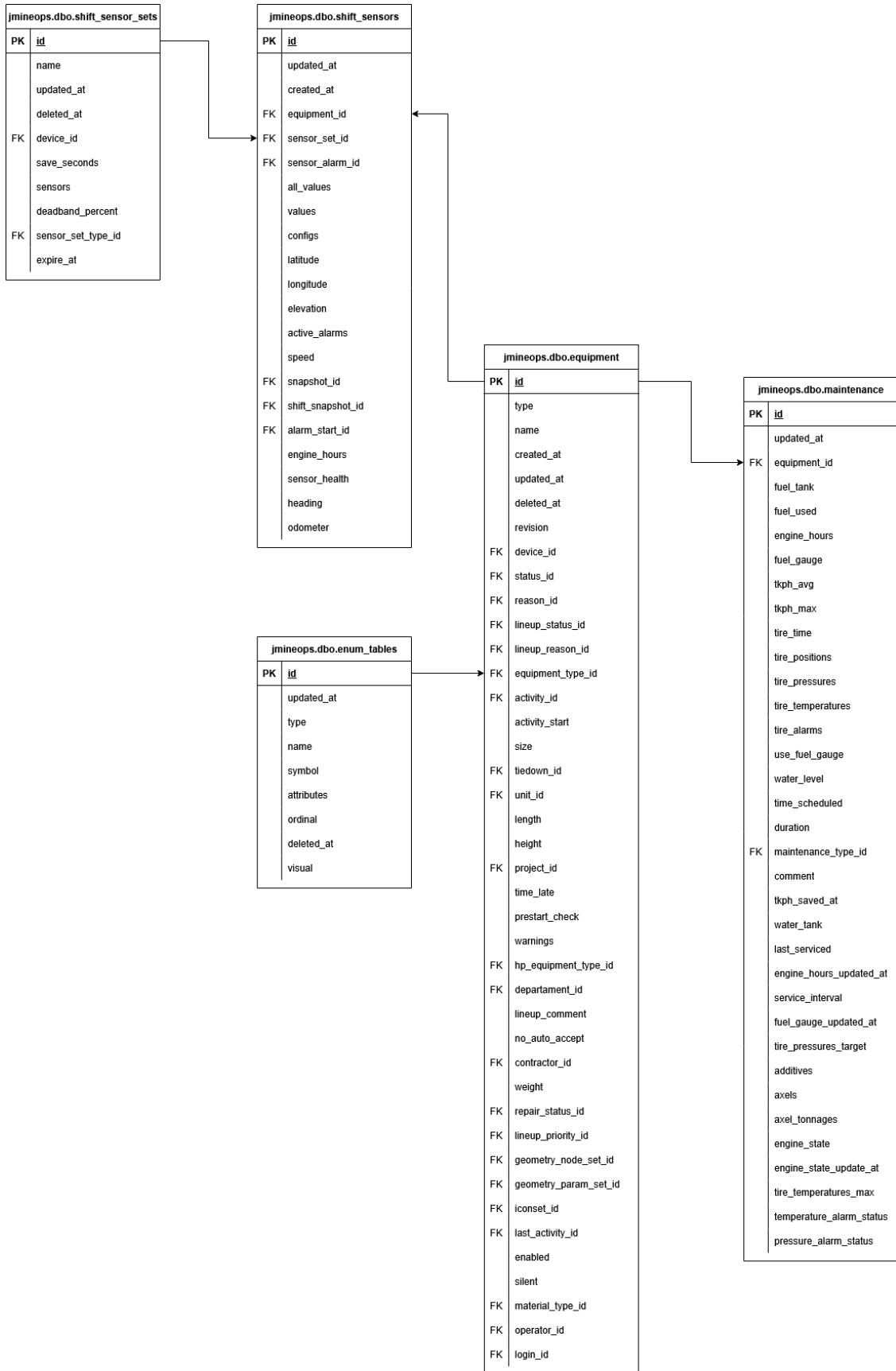
```
-- busca los datos minimos de sensores rpm y fuel para determinar los rangos de tiempo desde e
-- rpm
select min(created_at) from jmineops.dbo.shift_sensors
where created_at between '2024-01-01 11:00:00' and '2025-04-01 11:00:00'
and sensor_set_id = 490
-- resultado
-- 2025-02-04 21:44:00.000

-- fuel
select min(created_at) from jmineops.dbo.shift_sensors
where created_at between '2024-01-01 11:00:00' and '2025-04-01 11:00:00'
and sensor_set_id = 481
-- resultado
-- 2024-01-27 19:51:00.000
```

Otro aspecto crítico es la adaptación horaria: los datos se almacenan en UTC+0 pero requieren conversión a UTC-4 (horario boliviano) para alinearse con los turnos operativos. Los turnos se definen como diurnos (07:00 a 18:59) y nocturnos (19:00 a 06:59) según este huso horario local, garantizando coherencia en la clasificación de actividades.

Adicionalmente, se da la necesidad de implementar umbrales técnicos para diferenciar entre “ralentí” (motor en marcha sin movimiento) y “apagado”: intervalos mayores a cinco minutos sin actividad combinados con RPM menores a 300. Este criterio valida la integridad operativa de los registros al eliminar falsos positivos de inactividad.

Por consiguiente se tiene el siguiente diagrama que muestra los datos relacionados a las 3 variables críticas:



1.1.3 Obtencion del dataset

El procedimiento almacenado propuesto `[Model].[Sensor]` que obtendra los datos maneja dos escenarios temporales. Para fechas anteriores a febrero de 2025, se enfoca exclusivamente en el sensor de combustible, calculando el consumo en litros mediante la capacidad del tanque documentada en `dbo.maintenance` y clasificando el estado operativo (ralenti o movimiento) según la velocidad registrada. Para fechas posteriores, integra dinámicamente los datos de RPM mediante una tabla temporal, correlacionando las lecturas de combustible y RPM con precisión de segundos mediante la cláusula `OUTER APPLY`.

Un aspecto crítico es la adaptación horaria: los datos se almacenan en UTC+0, pero se convierten a UTC-4 (horario boliviano) para alinearse con los turnos operativos. Los turnos diurnos (07:00 a 18:59) y nocturnos (19:00 a 06:59) se definen según este uso local, asegurando coherencia en la clasificación de actividades. Adicionalmente, el modelo diferencia entre “ralenti” (motor en marcha sin movimiento) y “apagado” mediante umbrales técnicos: intervalos superiores a cinco minutos sin actividad y RPM menores a 300, validando así la integridad operativa de los registros.

Los resultados se estructuran en datasets con una nomenclatura estandarizada, como `T-243_fuel_2024-01`, que reflejan el equipo, el tipo de sensor y el periodo analizado. Esta estandarización facilita la trazabilidad y el consumo por herramientas de análisis predictivo, asegurando que los datos entregados representen condiciones operativas reales sin redundancias o sesgos temporales. La ejecución del procedimiento, mediante parámetros como `@FechaIni` y `@EquipmentID`, genera salidas técnicas listas para su implementación en modelos de machine learning, manteniendo un equilibrio entre precisión sensor y contexto operativo.

```
[ ]: ~~~sql
USE Msc_Dev
GO
CREATE OR ALTER PROCEDURE [Model].[Sensor]
    @FechaIni DATETIME,
    @FechaFin DATETIME,
    @EquipmentID INT
AS
BEGIN
    SET NOCOUNT ON;

    -- Ajustar a UTC+0 (los datos están en UTC)
    SET @FechaIni = DATEADD(HOUR, 4, @FechaIni);
    SET @FechaFin = DATEADD(HOUR, 4, @FechaFin);

    DECLARE @FuelTank DECIMAL(10,2);
    DECLARE @Model VARCHAR(15);
    DECLARE @EquipmentName VARCHAR(100);

    -- Obtener capacidad del tanque de combustible
    SELECT @FuelTank = fuel_tank
    FROM [jmineops].[dbo].[maintenance] WITH (NOLOCK)
    WHERE equipment_id = @EquipmentID;
```

```

-- Obtener modelo y nombre del equipo
SELECT
    @Model = et.name,
    @EquipmentName = e.name
FROM [jmineops].[dbo].[equipment] e WITH (NOLOCK)
LEFT JOIN [jmineops].[dbo].[enum_tables] et WITH (NOLOCK)
    ON e.equipment_type_id = et.id
WHERE e.id = @EquipmentID
AND et.name IN ('CAT 789C', 'CAT 793D');

-- Seleccionar según la fecha
-- busca los datos minimos de sensores rpm y fuel para determinar los
rangos de tiempo desde el que se tomaran en cuenta en los datasets
-- rpm, por esta razon es la condicion if
-- select min(created_at) from jmineops.dbo.shift_sensors
-- where created_at between '2024-01-01 11:00:00' and '2025-04-01 11:00:
00'
-- and sensor_set_id = 490
-- resultado
-- 2025-02-04 21:44:00.000
IF @FechaFin < '2025-02-04'
BEGIN
    -- SOLO SENSOR DE COMBUSTIBLE
    SELECT
        ShiftDate = CONVERT(DATE, DATEADD(HOUR, -4, f.created_at)),
        Shift = CASE
            WHEN CAST(DATEADD(HOUR, -4, f.created_at) AS TIME)
                BETWEEN '07:00:00' AND '18:59:59' THEN 'D'
            ELSE 'N'
        END,
        Timestamp = DATEADD(HOUR, -4, f.created_at),
        RecordDuration = DATEDIFF(SECOND, LAG(f.created_at) OVER(ORDER BY f.
created_at), f.created_at),
        Equipment = @EquipmentName,
        TruckFleet = ISNULL(@Model, 'NULL'),
        FuelLevel = ISNULL(f.[values], 0),
        FuelLevelLiters = ISNULL((@FuelTank * TRY_CAST(f.[values] AS
DECIMAL(10,2)) / 100), 0),
        FuelGauge = CASE
            WHEN TRY_CAST(f.[values] AS DECIMAL(5,2)) <= 10 THEN
'Critical'
            WHEN TRY_CAST(f.[values] AS DECIMAL(5,2)) <= 30 THEN
'Low'
            WHEN TRY_CAST(f.[values] AS DECIMAL(5,2)) <= 50 THEN
'Medium'

```

```

        WHEN TRY_CAST(f.[values] AS DECIMAL(5,2)) <= 80 THEN
↪ 'High'

        WHEN f.[values] IS NULL THEN 'NoData'
        ELSE 'Full'
    END,
    Speed = ISNULL(f.speed, 0),
    RPM = 0,
    Ralenti = CASE
        WHEN f.speed = 0 THEN 'Ralenti'
        WHEN f.speed > 0 THEN 'Moviendose'
        ELSE 'Otro'
    END,
    Latitude = f.latitude,
    Longitude = f.longitude,
    Elevation = f.elevation
FROM jmineops.dbo.shift_sensors f WITH (NOLOCK)
WHERE f.sensor_set_id = 481
    AND f.created_at BETWEEN @FechaIni AND @FechaFin
    AND f.equipment_id = @EquipmentID
ORDER BY f.created_at;
END
ELSE
BEGIN
    -- Crear tabla temporal en lugar de variable de tabla
    DROP TABLE IF EXISTS #TempRPMDData
    -- Tabla para almacenar resultados temporales
    CREATE TABLE #TempRPMDData (
        created_at DATETIME,
        equipment_id INT,
        [values] DECIMAL(10,2),
        speed DECIMAL(10,2)
    );

    -- Obtener datos de RPM usando el procedimiento más eficiente
    INSERT INTO #TempRPMDData
EXEC [Model].[explore_data]
    @Eq = @EquipmentID,
    @FechaIni = @FechaIni,
    @FechaFin = @FechaFin,
    @SensorSetId = 490;
    -- SENSOR DE COMBUSTIBLE + RPM
    WITH SensorFuel AS (
        SELECT
            created_at,
            equipment_id,
            [values],
            speed,

```

```

        latitude,
        longitude,
        elevation
FROM jmineops.dbo.shift_sensors WITH (NOLOCK)
WHERE sensor_set_id = 481
      AND created_at BETWEEN @FechaIni AND @FechaFin
      AND equipment_id = @EquipmentID
)
SELECT
    ShiftDate = CONVERT(DATE, DATEADD(HOUR, -4, f.created_at)),
    Shift = CASE
        WHEN CAST(DATEADD(HOUR, -4, f.created_at) AS TIME)
            BETWEEN '07:00:00' AND '18:59:59' THEN 'D'
        ELSE 'N'
    END,
    Timestamp = DATEADD(HOUR, -4, f.created_at),
    RecordDuration = DATEDIFF(SECOND, LAG(f.created_at) OVER(ORDER BY f.
↪created_at), f.created_at),
    Equipment = @EquipmentName,
    TruckFleet = ISNULL(@Model, 'NULL'),
    FuelLevel = ISNULL(f.[values], 0),
    FuelLevelLiters = ISNULL((@FuelTank * TRY_CAST(f.[values] AS
↪DECIMAL(10,2)) / 100), 0),
    FuelGauge = CASE
        WHEN TRY_CAST(f.[values] AS DECIMAL(5,2)) <= 10 THEN
↪'Critical'
        WHEN TRY_CAST(f.[values] AS DECIMAL(5,2)) <= 30 THEN
↪'Low'
        WHEN TRY_CAST(f.[values] AS DECIMAL(5,2)) <= 50 THEN
↪'Medium'
        WHEN TRY_CAST(f.[values] AS DECIMAL(5,2)) <= 80 THEN
↪'High'
        WHEN f.[values] IS NULL THEN 'NoData'
        ELSE 'Full'
    END,
    Speed = COALESCE(NULLIF(r.speed, 0), f.speed, 0),
    RPM = ISNULL(TRY_CAST(r.[values] AS DECIMAL(10, 2)), 0),
    Ralenti = CASE
        WHEN f.speed = 0 AND DATEDIFF(MINUTE, LAG(f.created_at) OVER
↪(ORDER BY f.created_at), f.created_at) BETWEEN 0 AND 5 THEN 'Ralenti'
        WHEN f.speed = 0 AND DATEDIFF(MINUTE, LAG(f.created_at) OVER
↪(ORDER BY f.created_at), f.created_at) > 5 THEN 'Apagado'
        WHEN f.speed > 0 THEN 'Moviendose'
        WHEN f.speed IS NULL AND DATEDIFF(MINUTE, LAG(f.created_at)
↪OVER (ORDER BY f.created_at), f.created_at) > 5 THEN 'Apagado'

```

```

        WHEN f.speed IS NULL AND DATEDIFF(MINUTE, LAG(f.created_at)
↪OVER (ORDER BY f.created_at), f.created_at) BETWEEN 0 AND 5 AND r.[values]
↪>= 300.0 THEN 'Ralenti'
        WHEN f.speed IS NULL AND DATEDIFF(MINUTE, LAG(f.created_at)
↪OVER (ORDER BY f.created_at), f.created_at) BETWEEN 0 AND 5 AND r.[values] <
↪300.0 THEN 'Apagado'
        ELSE 'Otro'
    END,
    Latitude = f.latitude,
    Longitude = f.longitude,
    Elevation = f.elevation
FROM SensorFuel f
OUTER APPLY (
    SELECT TOP 1
        [values],
        speed
    FROM #TempRPMDData r
    WHERE r.created_at <= f.created_at
    ORDER BY r.created_at DESC
) r
ORDER BY f.created_at;
-- Eliminar la tabla temporal al finalizar
DROP TABLE #TempRPMDData;
END
END;
GO

select * from jmineops.dbo.equipment

EXEC [Model].[Sensor]
    @FechaIni = '2025-08-01 07:00:00',
    @FechaFin = '2025-09-01 06:59:59',
    @EquipmentID = 311

select * from jmineops.dbo.shift_sensors
where equipment_id = 290
and sensor_set_id = 490
and created_at between '2025-02-01 08:00:00' and '2025-02-04 20:00:00'

```

1.2 Comprensión de datos de Sensores

1.2.1 Descripción del Dataset

Para garantizar consistencia en el análisis inicial, se priorizó una muestra focalizada en el camión T-210 durante marzo de 2025. Esta selección se fundamenta en:

Disponibilidad sensor completa: Es el primer período donde coinciden registros históricos de c

Representatividad operativa: El T-210 fue elegido por ser el primer equipo consistente en cuan

El dataset T-210_sensor_2025-03.csv contiene lecturas sensoras del período analizado, estructurado con las siguientes variables clave: FuelLevelLiters, RPM y Speed.

1.2.2 Series temporales

```
[ ]: from pathlib import Path
import polars as pl
import numpy as np
from datetime import datetime
from tabulate import tabulate

def load_sensor_data(data_path):
    """Carga y preprocesamiento inicial de datos de sensores con filtrado de
    ↪valores inválidos"""
    try:
        df = (
            pl.read_csv(
                data_path,
                separator=";",
                columns=[
                    "Timestamp",
                    "RecordDuration",
                    "FuelLevelLiters",
                    "FuelLevel",
                    "Speed",
                    "RPM",
                ],
            )
            .with_columns(
                pl.col("Timestamp").str.to_datetime(),
            )
            .sort("Timestamp")
            # Filtrado de valores negativos
            .filter(
                (pl.col("FuelLevelLiters") >= 0) &
                (pl.col("Speed") >= 0) &
                (pl.col("RPM") >= 0)
            )
            .with_columns([
                pl.col("FuelLevelLiters").diff().alias("DeltaFuel"),
                (
                    (pl.col("FuelLevelLiters") == 0)
                    & (pl.col("FuelLevelLiters").shift(-1) > 0)
                    & (pl.col("FuelLevelLiters").shift(1) > 0)
                )
            ])
```

```

        ).alias("SensorOff"),
        pl.col("FuelLevel").diff().alias("DeltaFuelPercent"),
    ])
)
return df

except FileNotFoundError:
    raise RuntimeError(f"Data file not found: {data_path}")

DATA_DIR = Path("../data-set/train_data_sensor/test_2025/
↳T-210_sensor_2025-03.csv")

if DATA_DIR.exists():
    data = load_sensor_data(DATA_DIR)
    print(f"Numero de registros, numero de columnas: {data.shape}")

    desc_df = data.describe()
    print("\nDescripción de columnas:")
    print(tabulate(
        desc_df.to_pandas(),
        headers="keys",
        tablefmt="grid",
        stralign="center",
        numalign="center"
    ))

    # Muestra de registros con tabulate
    print("\nMuestra de los 2 primeros registros:")
    print(tabulate(
        data.head(2).to_pandas(),
        headers="keys",
        tablefmt="grid",
        showindex=False,
        stralign="center",
        numalign="center"
    ))
else:
    print(f" Archivo no encontrado en: {DATA_DIR}")

```

1.2.3 Visualización de Series temporales

```

[ ]: from pathlib import Path
import polars as pl
import plotly.graph_objects as go
from datetime import date

# Ruta al archivo

```

```

DATA_DIR = Path("../..//data-set/train_data_sensor/test_2025/
↳T-210_sensor_2025-03.csv")

if DATA_DIR.exists():
    data = load_sensor_data(DATA_DIR)

    # Filtrar por el 2 de marzo de 2025
    filtered_data = data.filter(
        pl.col("Timestamp").dt.date() == date(2025, 3, 2)
    )

    if filtered_data.is_empty():
        print(" No hay datos para el 2 de marzo de 2025")
    else:
        fig = go.Figure()
        fig.add_trace(
            go.Scattergl(
                x=filtered_data["Timestamp"].to_numpy(),
                y=filtered_data["FuelLevelLiters"].to_numpy(),
                mode="lines",
                name="Nivel de Combustible",
                line=dict(color="#1f77b4", width=1),
            )
        )

        fig.update_layout(
            title="Nivel de Combustible - 2 de marzo de 2025",
            xaxis_title="Hora del dia",
            yaxis_title="Litros",
            hovermode="x unified",
            template="plotly_white",
            height=500,
        )

        fig.show()
else:
    print(f" Archivo no encontrado en: {DATA_DIR}")

```

Observaciones Analizando la grafica lo primero que observamos son los picos elevandos y fluctuantes en el tiempo, esto tiene un motivo, el sensor de combustible es uno que mide el nivel del tanque, entonces al moverse el camion este puede variar demasiado, de entre picos de 10 a 20 positivamente y de -20 a -10, esos picos obteniendo el delta de combustible, entonces estos movimientos generados al moverse el camion pueden generar variones de entre 20 a -20 litros, esto se demuestra tomando un histograma con rangos de 10 en 10 obtenidos con el metodo sqrt, con unos bins de 665 en este caso, simplemente aplicando el siguiente codigo: `if method == "sqrt": return int(np.sqrt(n))`, probando algunos otros metodos de distribucion del delta de combustible o en otras palabras la diferencia de combustible se tienen que sqrt nos muestra que en ese rango

se encuentran la mayor parte de estas fluctuaciones, otros metodos a destacar usados en la experimentacion consiste en aplicar : sqrt, auto, fd, scott, sturges, sqrt. pero sqrt resutlo mejores resultados en cuanto a buscar estas fluctuaciones mas comunes con un total estimado de 85.709 registros de rangos 10 a 20 y de 102.785 registros de rangos de -10 a -20 entonces podemos decir que ± 20 son las fluctuaciones mas aceptables.

1.2.4 Histogramas de delta de combustible

```
[ ]: from pathlib import Path
import polars as pl
import numpy as np
import plotly.express as px
from datetime import datetime
from typing import Dict, Any

def load_sensor_data(data_path: Path) -> pl.DataFrame:
    """Carga y preprocesamiento básico de datos"""
    try:
        df = (
            pl.read_csv(
                data_path,
                columns=["TimeStamp", "RecordDuration", "FuelLevelLiters", "FuelLevel", "Speed", "RPM"],
                schema_overrides={"RPM": pl.Float64}
            )
            .with_columns(
                pl.col("TimeStamp").str.to_datetime(),
            )
            .sort("TimeStamp")
            .with_columns(
                pl.col("FuelLevelLiters").diff().alias("DeltaFuel"),
                (
                    (pl.col("FuelLevelLiters") == 0)
                    & (pl.col("FuelLevelLiters").shift(-1) > 0)
                    & (pl.col("FuelLevelLiters").shift(1) > 0)
                ).alias("SensorOff"),
                pl.col("FuelLevel").diff().alias("DeltaFuelPercent"),
            )
        )
        return df
    except FileNotFoundError:
        raise RuntimeError(f"Archivo no encontrado: {data_path}")

def generate_column_stats(df: pl.DataFrame, column: str) -> Dict[str, Any]:
    """Genera estadísticas básicas para una columna numérica"""
    if column not in df.columns:
        raise ValueError(f"Columna {column} no encontrada en el DataFrame")
```

```

col_data = df[column]
q1 = col_data.quantile(0.25)
q3 = col_data.quantile(0.75)

return {
    "min": col_data.min(),
    "max": col_data.max(),
    "mean": col_data.mean(),
    "median": col_data.median(),
    "std_dev": col_data.std(),
    "q1": q1,
    "q3": q3,
    "iqr": float(q3 - q1) if q1 and q3 else None,
    "count": col_data.count()
}

def calculate_bins(stats: Dict[str, Any], method: str = "auto", bins: int = None) -> int:
    """Calcula número de bins usando diferentes métodos"""
    if bins:
        return bins if bins > 0 else 10

    n = stats["count"]
    if n == 0:
        return 0

    method = method.lower()
    if method == "sqrt":
        return int(np.sqrt(n))
    if method == "sturges":
        return int(np.ceil(np.log2(n) + 1))
    if method == "fd":
        return freedman_diaconis_bins(stats["iqr"], n, stats["max"] - stats["min"])
    if method == "scott":
        return scott_bins(stats["std_dev"], n, stats["max"] - stats["min"])

    # Método auto por defecto
    return freedman_diaconis_bins(stats["iqr"], n, stats["max"] - stats["min"])

def freedman_diaconis_bins(iqr: float, n: int, data_range: float) -> int:
    """Cálculo de bins por Freedman-Diaconis"""
    if iqr == 0 or n == 0:
        return 1
    bin_width = 2 * iqr / (n ** (1/3))
    return max(1, int(np.ceil(data_range / bin_width)))

```

```
def scott_bins(std_dev: float, n: int, data_range: float) -> int:
    """Cálculo de bins por Scott"""
    if std_dev == 0 or n == 0:
        return 1
    bin_width = 3.5 * std_dev / (n ** (1/3))
    return max(1, int(np.ceil(data_range / bin_width)))
```

1.2.5 Visualización de histograma de delta de combustible

```
[ ]: def plot_histogram(df: pl.DataFrame, column: str, bins: int):
    """Visualiza el histograma usando Plotly"""
    if column not in df.columns:
        raise ValueError(f"Columna {column} no existe en el DataFrame")

    fig = px.histogram(
        x=df[column].to_numpy(),
        nbins=bins,
        title=f"Distribución de {column}",
        labels={"x": column, "y": "Frecuencia"},
        template="plotly_white"
    )

    fig.update_layout(
        bargap=0.1,
        hovermode="x unified",
        xaxis_title=column,
        yaxis_title="Frecuencia"
    )

    fig.show()

if __name__ == "__main__":
    # Ejemplo de uso
    DATA_PATH = Path("../data-set/val_data_sensor/T-210_sensor.csv")

    if not DATA_PATH.exists():
        print(f" Archivo no encontrado: {DATA_PATH}")
    else:
        # Cargar datos
        df = load_sensor_data(DATA_PATH)

        # Seleccionar columna para análisis
        target_column = "DeltaFuel"
```

```
# Generar estadísticas
stats = generate_column_stats(df, target_column)
for key, value in stats.items():
    print(f"{key}:{value}")

# Calcular bins
n_bins = calculate_bins(stats, method="sqrt")
print(f"Número de bins calculados: {n_bins}")

# mostrar la grafica
plot_histogram(df, target_column, n_bins)
```

1.2.6 Descripción del dataset de sensores